Information Security, Assignment 1

RekamKu: An Encrypted Personal Health-Record Vault

Group 10 Kartika Nana Naulita (5025241021), Safa Mashita (5025241022), Devina Balqis Aurora (5025241034), Via Hana Nurma Putri (5025241048)
Repository: https://github.com/mashitaaa/RekamKu | Website : https://rekamku.pythonanywhere.com

1. Scenario and why the application is useful

Patients with chronic conditions such as epilepsy, diabetes or hypertension are treated in several places: a puskesmas, a referral hospital, a private laboratory. Their lab results, discharge summaries, the KTP and BPJS scans requested at registration, and the medical videos doctors ask for (for example a seizure filmed by a family member) end up on paper or in a phone gallery, where a lost phone, an automatic cloud backup or a message sent to the wrong chat group can expose a diagnosis. RekamKu is the patient's own record folder. Its users are patients, or the carer who manages their account. They store a health profile, identity scans, PDF/DOCX/XLSX medical documents and videos, and open or download them at the next consultation. Because the records belong to the patient, the requirement that only the authenticated user gets their own data back, and gets it back intact, matches the scenario exactly. Law No. 17/2023 on Health gives patients the right to access the information in their medical record (Art. 276(e)); RekamKu is where they keep that information.

2. Which data is private (Law No. 27/2022 on Personal Data Protection, UU PDP; GDPR)

Data	Class	Basis and consequence
Diagnoses, allergies, medication, blood type, BPJS number, medical documents, medical videos	Specific	Health data, including use of health services (UU PDP Art. 4(2)(a)); special category under GDPR Art. 9(1).
KTP and BPJS card scans	Specific (handled as)	Name, NIK and address are general data. The face photo is potentially biometric: the elucidation of Art. 4(2)(b) gives facial images as an example when they allow unique identification, while under the GDPR a photo is biometric only when processed by specific technical means (Recital 51), which RekamKu does not do. The BPJS card reveals use of health services (Art. 4(2)(a)). The folder is treated as specific.
Name, NIK, date of birth, sex, address, phone, emergency contact (a third party)	General	Art. 4(3)(a)(b)(f). Combined with a medical record they identify the patient together with the illness.
Random IDs, username HMAC, salts and KDF parameters, log public key, ciphertext size	Pseudonymous technical data	Still personal data, because each value is linked to one account (GDPR Recital 26). Left readable only because sign-in needs them before any user key exists; none of them reveals content.

Decision: all personal data, general and specific, is encrypted, because general data next to health data is just as revealing. The classification still sets the legal basis, the labels in the UI and the /privacy page. The basis we rely on is explicit consent, one of the six bases in Art. 20(2) (GDPR Art. 9(2)(a)), given through a mandatory checkbox at registration. Data-subject rights are features: download (Art. 7), edit the profile (Art. 6), delete a file or the account (Art. 8) and an activity log (Art. 16(2)(h)).

3. Design decisions and their justification

Decision	Choice	Reason (and rejected alternatives)
Reading "protect everything with AES, DES and RC4"	Cascade: plaintext → DES-CBC → RC4 → AES-256-GCM, a separate HKDF-derived key per layer	Three copies, one per cipher, rejected: an attacker only needs the weakest copy (56-bit DES) and storage triples. One cipher per file rejected: some records would be protected by RC4 alone. In the cascade every byte passes through all three ciphers.
Layer order	AES-GCM outermost	The GCM tag is checked before RC4 or DES touch the data (encrypt-then-MAC), so there is no DES-CBC padding oracle and no unverified plaintext is released.
Modes (no ECB)	AES: GCM, nonce = chunk index. DES: CBC, random IV per chunk. RC4: stream cipher	ECB rejected: equal blocks give equal ciphertext, which leaks patterns in scans and repeated form fields. GCM gives confidentiality and integrity in one pass; counter nonces are safe because each object has its own AES key (HKDF, random 24-byte salt). DES-CTR rejected: a repeated nonce repeats the keystream, the same failure as RC4. Sweet32 needs ~2^32 blocks (32 GB) under one key; a file is at most 200 MB.
RC4	RC4-drop[3072], per-chunk key = HMAC(k, index)	Discards the most biased start of the keystream. A key is never reused, because reuse leaks the XOR of two plaintexts (shown in E6).
File format	1 MiB chunks; AAD = H(header‖owner‖id) ‖ index ‖ last flag	Constant server memory and seekable video (HTTP Range). Each chunk is verified before its bytes are sent; if a later chunk fails, the transfer is aborted, so the client may receive an incomplete download but never unverified data. The AAD detects modification, reordering, truncation and swapping files between users.
Key handling	Envelope: password →scrypt+pepper→ KEK → UMK → per-file FEK	A password change re-wraps only the UMK. Deleting a file removes its wrapped FEK and blob from the live system; a database backup taken earlier still holds the wrapped FEK, so deletion does not reach backups. The pepper comes from the server key; it stops offline guessing on a stolen database only if that key is kept elsewhere (REKAMKU_SERVER_KEY); by default it lies on the same disk (instance/server.key). A 100-bit recovery code wraps the same UMK, so a forgotten password does not erase medical records.
Password KDF	scrypt N=2^17, r=8, p=1	Memory-hard (128 MiB per guess). Measured 589 ms per sign-in (E7), an acceptable wait; N=2^15 takes 163 ms and is ≈3.6× cheaper to guess.
Where encryption runs	Server side, keys derived from the user's password	Browser-side encryption rejected: WebCrypto has no DES or RC4, and the server must check file types and serve Range requests. Covered: stolen database, disk or backup, a curious storage admin, files modified at rest. Not covered: a compromised running server.
Metadata and activity log	File metadata inside the encrypted envelope; log entries sealed to the user's X25519 key	File names, categories, sizes and dates are encrypted. A failed sign-in must be logged while the vault is locked, so each entry uses a one-time X25519 key and the same cascade; only the owner's private key (wrapped by the UMK) opens it. A test checks every database column.

Architecture, storage and access	Flask web app in one process; SQLite + randomly named blobs; session, CSRF and owner check on every query	One process, because session keys live in RAM; SQLite is enough for a personal vault and needs no separate database server to secure, since every personal column is encrypted anyway. Uploads are a raw streamed body, because Werkzeug spills multipart files >500 KB to /tmp in the clear; writes are atomic. The UMK stays in RAM (the cookie holds a random ID), 15-min idle timeout, five sign-in attempts per 15 min. The AAD also binds each file to its owner.

4. Analysis: what we measured, why it is the right analysis here, and what it means

We compared AES, DES and RC4 on the things that matter to a patient and to a small server: how long it takes to encrypt and decrypt the files patients actually keep (from a small spreadsheet to a three-minute video), how much of that time each cipher takes inside the cascade, how much server memory large videos need, whether a silent change to a stored lab value gets caught, and how much extra storage encryption adds. We chose these because they follow the real risks of this scenario. A patient waits at every upload and at the next consultation, many patients share one small server, and a wrong lab value can lead to wrong treatment. A speed-only benchmark, or a discussion of key lengths, would miss all of that. For the method, the data was loaded into memory first so disk speed does not affect the results. Each test had two warm-up runs and then many repetitions (40 for small files, 12 and 7 for the two videos), with the cipher order shuffled every round and the garbage collector switched off. We report the median and the spread (IQR). Everything ran on an Intel Core i7-13620H laptop with 15.7 GB RAM, an NVMe SSD, Windows 11 and AES-NI available, using Python 3.13.3 and pyca/cryptography 50.0.2. The whole suite was run twice on the final code and the results stayed close. AES-GCM varied the most, simply because its times are so short that timer noise counts for more. The gaps between the ciphers are far larger than this variation (for the 90 MB video, DES is about 70 times slower than AES-GCM), so the conclusions do not depend on it. Figure 1 and all numbers come from the first run, and absolute times will differ on other machines. Scripts and raw CSV files are in bench/.

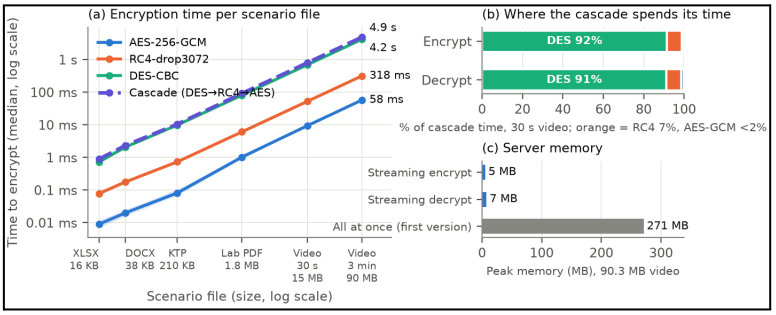


Figure 1. (a) Median encryption time (band = IQR); decryption is similar. (b) Share of each layer in the cascade, 30 s video. (c) Peak Python heap (tracemalloc) while encrypting the 90 MB video; "all at once" is a whole-file baseline re-implemented in the benchmark script.

- **DES is the expensive part, AES-GCM is almost free.** AES-256-GCM runs in the range of gigabytes per second, RC4 in the hundreds of megabytes per second, and DES only at around 20 MB/s. Inside the cascade, DES takes over 90% of the time, RC4 a small slice, and AES-GCM about 1%. One detail: pyca only offers DES as 3DES-EDE with three identical keys. It gives the same output as single DES (it passes the FIPS 81 test vector) but needs three passes, so real single DES would be up to about three times faster. Even then, DES would still take most of the cascade time.
- **Patients barely notice the cost.** Encrypting a KTP scan takes around 10 ms and a 1.8 MB lab PDF about a tenth of a second, and a full upload through the app adds only a little on top (Flask test client, no network or TLS). A three-minute video needs about five seconds to encrypt and about the same to decrypt. Uploading that video over an assumed 10 Mbit/s connection takes more than a minute, so encryption adds only about 7% to the wait. Thanks to chunking, jumping to the middle of a video takes roughly a tenth of a second whether the file is 15 MB or 90 MB, while decrypting the whole 90 MB file takes several seconds.
- **Server capacity is limited by DES, not AES.** One core is busy for about five seconds per three-minute video, almost all of it in DES. With AES-GCM alone the same video would take well under a tenth of a second of cipher time. Streaming keeps memory use at a few megabytes, whereas the whole-file baseline needs 271 MB, three times the size of the video.
- **Integrity in practice (E6).** Without the key, an attacker who can write to storage changes “Fasting glucose: 95” into “35”. With RC4 alone the result decrypts to exactly “35 mg/dL” and nobody notices. With DES-CBC alone it also passes, and only the preceding block comes out garbled. AES-GCM and the cascade both reject it. Since a silently altered lab value can lead to wrong treatment, the main job of AES-GCM here is integrity. Reusing an RC4 key also leaked “Diagnosis: epilepsy” from the XOR of two ciphertexts, which is why RC4 keys are made per chunk. Storage overhead is negligible: under half a percent for the smallest file and practically nothing for the video.
- **Conclusion.** Confidentiality and integrity rest on AES-256-GCM, which is also the cheapest layer. DES (56-bit key, broken by the EFF in 56 hours in 1998) and RC4 (keystream biases, prohibited in TLS by RFC 7465) are included only because the assignment requires them. They sit inside the GCM layer, so the design does not depend on their strength. We do not recommend them for production. Outside this assignment we would use AES-256-GCM alone, which would also remove most of the measured cost.

5. Limitations

The server sees plaintext while a session is active (not end-to-end), so a compromised running server remains a risk; session keys in RAM limit the app to one process, and Python cannot guarantee that memory is wiped. Ciphertext sizes are visible. The AAD carries no version, so someone with write access to storage could restore an older valid ciphertext of the same record (rollback) without detection. Consent is required at registration but not stored with a timestamp, although Art. 22(1) requires written or recorded consent. Losing both the password and the recovery code makes the data unrecoverable, a deliberate trade-off. Testing: 49 automated tests (official vectors, IDOR, bit flips, truncation, blob swapping, CSRF, no plaintext in the database or on disk), all passing.